

Informe del módulo de recuperación

CSDI RAG Engine

1 Módulo de recuperación

El módulo de recuperación es el componente central del sistema. Su responsabilidad es, dado el texto libre de una consulta, identificar y ordenar los fragmentos del corpus que mayor relevancia tienen para responderla. El módulo no es un recuperador único: es una cadena de etapas especializadas que transforma la consulta original en un conjunto de fragmentos de alta calidad listos para ser entregados al generador de respuestas.

Para que un sistema RAG funcione correctamente, no es suficiente recuperar documentos que compartan palabras con la consulta: es necesario recuperar los fragmentos más relevantes independientemente de cómo esté formulada la pregunta, del idioma en que esté escrita y de si los términos exactos aparecen o no en el corpus. BM25 aplicado de forma aislada no puede satisfacer esas condiciones simultáneamente. Un recuperador puramente léxico como BM25 exige que los términos de la consulta coincidan con los del documento tras normalización morfológica; falla ante paráfrasis, consultas en un idioma distinto al del corpus o preguntas conceptuales sin vocabulario técnico exacto. Un recuperador puramente vectorial resuelve esas brechas lingüísticas pero pierde precisión ante identificadores exactos como nombres de funciones, mensajes de error o versiones de API, que los modelos de embeddings proyectan cerca de términos relacionados aunque la distinción exacta sea la que el usuario necesita. Ninguno de los dos enfoques por sí solo es suficiente para un sistema RAG de calidad en documentación técnica multilingüe. El módulo de recuperación resuelve esta tensión combinando ambas señales y añadiendo una fase de reordenamiento de alta precisión.

Diseño general del flujo de recuperación. El flujo completo de recuperación se organiza en tres etapas ordenadas: *expansión opcional de la consulta*, *recuperación híbrida* y *reordenamiento*. Esta organización no es arbitraria: cada etapa actúa sobre la salida de la anterior y permite que los fragmentos entregados al generador sean los más relevantes que el corpus puede ofrecer.

La expansión transforma la consulta original en una representación más rica antes de hacer la búsqueda vectorial. La recuperación híbrida combina dos señales complementarias, léxica y semántica, en una lista única de candidatos. El reordenamiento evalúa cada par consulta-fragmento de forma conjunta para mejorar la precisión del ordenamiento final.

Componentes del recuperador léxico: BM25. El primer componente del recuperador híbrido es **BM25Retriever**. Este componente carga en memoria, al arrancar la aplicación, el índice invertido construido por el módulo de indexación: el diccionario de términos, las listas de postings, las longitudes documentales y las estadísticas globales del corpus. Cuando recibe una consulta, aplica exactamente la misma tokenización usada durante la indexación, consulta las listas de postings de los términos presentes y calcula la puntuación BM25 de cada documento candidato.

El recuperador léxico aporta una ventaja concreta en el dominio del sistema. La documentación técnica contiene nombres de funciones, clases, mensajes de error, operadores y palabras clave del lenguaje que rara vez son parafraseados: quien busca `asyncio.gather` o `Array.prototype.flatMap`

espera encontrar exactamente esas cadenas en los documentos recuperados. BM25 aprovecha esa propiedad porque compara términos directamente después de normalización morfológica, sin proyectarlos a un espacio de representación que podría absorber distinciones relevantes.

Componentes del recuperador semántico: embeddings y FAISS. El segundo componente es `VectorRetriever`. Utiliza el modelo de embeddings `BAAI/bge-m3`, un modelo multilingüe que produce vectores de 1024 dimensiones. Esta elección no fue la inicial: durante el desarrollo se trabajó primero con un modelo de un solo idioma que producía vectores de 384 dimensiones. Las pruebas con ese modelo mostraron que la calidad de la recuperación caía de forma notable cuando el usuario formulaba consultas en español sobre documentación escrita en inglés, ya que el modelo no tenía representaciones compartidas entre ambos idiomas. Al cambiar a `BAAI/bge-m3`, que fue entrenado sobre datos en más de cien idiomas, esa brecha desapareció: la consulta en español y el fragmento en inglés quedan proyectados a regiones cercanas del espacio de representación cuando describen el mismo concepto. El costo de esta mejora es un vector tres veces más grande (1024 frente a 384 dimensiones), lo que aumenta el uso de memoria del índice y el tiempo de codificación, pero se consideró un intercambio aceptable dado el perfil multilingüe real del sistema.

Los vectores de los fragmentos se almacenan en un índice FAISS de tipo HNSW (*Hierarchical Navigable Small World*). HNSW es una estructura de grafo jerárquico que permite realizar búsqueda aproximada de vecinos más cercanos con muy bajo costo de consulta. A diferencia de índices planos que comparan el vector de consulta contra todos los vectores del corpus, HNSW navega capas del grafo reduciendo el número de comparaciones necesarias. Los parámetros de construcción `HNSW_M=32` y `HNSW_EF_CONSTRUCTION=200` controlan la densidad del grafo y la precisión de la indexación; el parámetro de búsqueda `HNSW_EF_SEARCH=50` controla el equilibrio entre velocidad y exhaustividad en tiempo de consulta.

La métrica de similaridad utilizada es el producto interno, que equivale a similaridad coseno cuando los vectores están normalizados en norma L2. Los embeddings se normalizan al momento de la indexación y al momento de codificar la consulta, garantizando coherencia entre ambas fases. Cuando se recibe una consulta, `VectorRetriever` la codifica, consulta el índice FAISS y filtra automáticamente los documentos marcados como eliminados antes de devolver resultados.

Elección de HNSW sobre índices alternativos. La decisión de usar HNSW en lugar de un índice plano o una estructura basada en árboles responde a la escala esperada del corpus. Un índice plano en FAISS compara el vector de consulta contra todos los vectores en tiempo lineal, lo cual es exacto pero impracticable cuando el corpus tiene decenas o cientos de miles de fragmentos. Los índices de cuantización como IVF reducen ese costo dividiendo el espacio en celdas, pero requieren entrenamiento previo sobre el corpus y pueden perder calidad en corpus pequeños o poco densos.

HNSW no requiere entrenamiento, escala de forma eficiente con el tamaño del corpus y mantiene una alta tasa de recuperación en la práctica. Para el dominio de documentación técnica, donde la precisión en los primeros resultados es crítica, esta estructura es más adecuada que alternativas que sacrifican calidad para ganar velocidad.

Fusión híbrida mediante Reciprocal Rank Fusion. El componente central del módulo es `HybridRetriever`. Este componente ejecuta `BM25Retriever` y `VectorRetriever` de forma concurrente usando un `ThreadPoolExecutor`, reúne ambas listas de resultados y las combina mediante *Reciprocal Rank Fusion* (RRF).

RRF no utiliza directamente las puntuaciones de cada recuperador, sino únicamente el rango de cada documento dentro de su lista. La puntuación combinada de un documento d se calcula como:

$$\text{score}_{\text{RRF}}(d) = \sum_{r \in R} w_r \cdot \frac{1}{k + \text{rank}_r(d)}$$

donde R es el conjunto de recuperadores activos, w_r es el peso asignado al recuperador r , $\text{rank}_r(d)$ es la posición de d en la lista del recuperador r (empezando en 1), y k es una constante de suavizado con valor por defecto $k = 60$. Si un documento no aparece en la lista de un recuperador, no contribuye a la suma por ese recuperador.

Los pesos por defecto son `bm25_weight=0.3` y `vector_weight=0.7`. Esta distribución reconoce que la similitud semántica es la señal más general, pero no descarta el aporte de la coincidencia léxica para términos técnicos. Ambos pesos son configurables en tiempo de ejecución mediante `update_weights()`, lo que permite ajustar el comportamiento del recuperador sin reiniciar la aplicación.

Por qué RRF y no combinación directa de puntuaciones. Combinar directamente las puntuaciones de BM25 y del recuperador vectorial plantea un problema de escala. BM25 produce puntuaciones cuya magnitud depende del tamaño del corpus, la frecuencia de los términos y la longitud de los documentos; los scores de FAISS producen valores de producto interno que dependen de la distribución del modelo de embeddings. Estas magnitudes son heterogéneas y no comparables sin una normalización previa que puede ser sensible a la distribución específica de cada consulta.

RRF evita este problema por completo: solo considera el orden, no el valor. Un documento que aparece bien posicionado en ambas listas obtendrá una puntuación alta independientemente de la escala de cada recuperador. Además, RRF es robusto ante casos donde un recuperador devuelve pocos resultados o donde las puntuaciones están muy concentradas en pocos documentos. Por estas razones, RRF es una elección técnicamente sólida para la fusión híbrida sin requerir calibración por corpus.

Expansión de consulta mediante HyDE. Para mejorar la calidad de la búsqueda vectorial en consultas breves o abstractas, el sistema implementa *Hypothetical Document Embeddings* (HyDE). Cuando está habilitado, antes de codificar la consulta con el modelo de embeddings, el pipeline solicita al modelo generativo que produzca un documento hipotético que respondería la consulta. Ese documento hipotético, en lugar del texto de la consulta original, se usa como entrada al modelo de embeddings para la búsqueda vectorial.

La motivación técnica es que las consultas y los documentos tienen distribuciones textuales distintas: una consulta es corta y directiva mientras que un documento técnico es extenso y descriptivo. El modelo de embeddings produce vectores para ambos, pero un documento hipotético está más cerca, en ese espacio de representación, a los documentos reales del corpus que la consulta original. HyDE se aplica únicamente sobre el componente vectorial; BM25 continúa operando sobre la consulta original porque la coincidencia léxica no se beneficia de esta expansión.

Si la expansión falla por cualquier razón, el sistema retrocede a la consulta original sin interrumpir el flujo.

HyDE está desactivado por defecto por una razón de diseño que va más allá de la latencia: el sistema establece como principio que el modelo generativo interviene únicamente en la etapa de generación de respuesta. Usar el LLM dentro del pipeline de recuperación rompe esa separación, convirtiendo una etapa que debería ser determinista y auditable en una etapa dependiente de un modelo externo. Esto introduce un punto adicional de fallo, aumenta el costo por consulta y hace más difícil aislar la causa cuando la recuperación produce resultados incorrectos.

Sin embargo, HyDE se mantiene en el sistema porque ilustra de forma concreta cómo una intervención mínima del LLM —generar un único párrafo antes de la búsqueda vectorial— puede mejorar la calidad de la recuperación sin modificar ningún componente del índice ni del pipeline. Es un ejemplo observable de la tensión entre pureza arquitectónica y rendimiento práctico, y su presencia como opción configurable permite evaluar esa diferencia en el corpus real. Se activa mediante la variable de entorno `HYDE_ENABLED=true`.

Reordenamiento mediante Cross-Encoder. La recuperación híbrida produce una lista de candidatos ordenados por su puntuación RRF. Sin embargo, el ranking RRF se basa en el orden producido por dos modelos de primera etapa que procesan consulta y documento de forma independiente. Un *cross-encoder* puede evaluar cada par (consulta, fragmento) conjuntamente, capturando interacciones entre ellos que los encoders independientes no modelan.

El reordenador implementado es `CrossEncoderReranker`, que utiliza el modelo `cross-encoder/ms-marco-MiniLM`. La elección de este modelo responde a tres criterios. Primero, fue entrenado sobre MS-MARCO, un benchmark de 8,8 millones de pares (consulta, pasaje) extraídos de búsquedas web reales, que es el conjunto de entrenamiento estándar para sistemas de ranking de pasajes. Segundo, la arquitectura MiniLM de 6 capas ofrece un compromiso entre calidad y latencia: con aproximadamente 22 millones de parámetros, puede puntuar los 30 pares en CPU sin introducir una demora apreciable. Tercero, cross-encoders más grandes, como `ms-marco-electra-base`, ofrecen mayor precisión de ranking pero multiplican el tiempo de inferencia, lo que no está justificado para un conjunto de 30 candidatos ya preseleccionados por la recuperación híbrida.

En la primera etapa del pipeline se recuperan `reranker_candidate_k=30` candidatos; el reordenador puntúa los 30 pares (consulta, fragmento) y devuelve los `context_chunks=15` mejor posicionados. La relación 2:1 entre candidatos y fragmentos finales es una proporción deliberada: recuperar menos candidatos reduciría la cobertura del corpus expuesta al cross-encoder; recuperar más incrementaría el costo de inferencia sin garantizar una mejora proporcional en la calidad del conjunto final.

El uso de una primera etapa de candidatos amplia seguida de un reordenador preciso es una estrategia estándar en recuperación de información moderna. El cross-encoder es computacionalmente más costoso que los encoders de primera etapa, por lo que no puede aplicarse sobre todo el corpus directamente, pero puede aplicarse eficientemente sobre un conjunto reducido de candidatos ya filtrados.

Por qué BM25 solo no es suficiente para un sistema RAG. Un sistema RAG exige que el módulo de recuperación entregue al generador los fragmentos con mayor densidad de información relevante para la consulta. Si esos fragmentos no son los correctos, el generador producirá una respuesta incorrecta o incompleta sin importar su capacidad. BM25 por sí solo no puede garantizar esa calidad de recuperación en las condiciones reales del sistema por tres razones estructurales.

Primero, BM25 es sensible al vocabulario: una consulta formulada en español sobre documentación en inglés no encuentra coincidencias léxicas en el corpus, devolviendo resultados vacíos o irrelevantes aunque los documentos necesarios existan. Segundo, BM25 no captura relaciones semánticas: una consulta como “cómo iterar sobre una lista” no recuperará un fragmento que explica `forEach` si ese término no aparece en la consulta. Tercero, incluso cuando BM25 recupera los documentos correctos, los ordena por frecuencia de término, no por relevancia semántica al contexto completo de la consulta, lo que degrada la calidad del conjunto enviado al generador.

La recuperación vectorial resuelve el primer y segundo problema pero introduce el suyo propio: colapsa la distinción entre identificadores exactos. La fusión híbrida resuelve ambos problemas a la vez. El reordenamiento con cross-encoder corrige el tercero. Cada componente es necesario para

cumplir el requisito de un sistema RAG funcional; omitir cualquiera de ellos introduce una categoría de fallo que los demás no pueden compensar.

Integración en el pipeline RAG. El módulo de recuperación está integrado en `RAGPipeline`, que coordina la secuencia completa desde la consulta del usuario hasta la respuesta generada. El pipeline invoca primero a `HybridRetriever` para obtener candidatos, opcionalmente a través de la expansión HyDE. Si el reordenador está activo, reordena los candidatos antes de seleccionar los fragmentos finales. Con esos fragmentos construye el prompt y lo envía al modelo generativo.

El resultado devuelto al cliente incluye la respuesta generada, las fuentes utilizadas con sus URLs y títulos, el modelo utilizado y los conteos de tokens. Esta trazabilidad es parte del diseño del módulo: el usuario puede verificar de dónde proviene la información y el sistema puede auditar qué fragmentos influyeron en cada respuesta.

API del módulo de recuperación. El módulo expone varios endpoints dentro del servidor. El endpoint `POST /api/v1/search` ejecuta la búsqueda híbrida directamente y devuelve los fragmentos recuperados con sus metadatos de fuente, título, breadcrumb y texto. El endpoint `POST /api/v1/search/bm25` expone el recuperador léxico de forma aislada, útil para diagnóstico y comparación. El endpoint `POST /api/v1/vector/search` expone el recuperador vectorial de forma aislada. El endpoint `POST /api/v1/rag/query` ejecuta el pipeline completo incluyendo generación de respuesta.

La existencia de endpoints individuales para cada componente del recuperador permite validar de forma independiente cada señal de relevancia y diagnosticar casos donde un enfoque aporta más que el otro. Esto fue una decisión de diseño tomada durante el desarrollo para facilitar la depuración y el ajuste de pesos de fusión.

Configuración y parámetros clave. Los parámetros del módulo de recuperación se controlan mediante variables de entorno y son ajustables en tiempo de ejecución a través de la API de configuración. Los parámetros más relevantes son el número de candidatos de primera etapa para el reordenador (`RERANKER_CANDIDATE_K=30`), el número de fragmentos finales enviados al generador (`LLM_CONTEXT_CHUNKS=15`), los pesos de fusión BM25 y vectorial (`bm25_weight=0.3`, `vector_weight=0.7`) y el parámetro de suavizado de RRF ($k = 60$).

Este diseño de configuración externalizada permite adaptar el comportamiento del recuperador a condiciones cambiantes del corpus sin modificar el código. Si el corpus crece o cambia de dominio, los pesos de fusión y el tamaño del conjunto de candidatos pueden ajustarse de forma controlada y sin interrumpir el servicio.

Justificación de los valores de los parámetros. Los valores por defecto de los parámetros del módulo no fueron elegidos arbitrariamente. Cada uno responde a una propiedad del dominio o a evidencia empírica de la literatura.

El parámetro de suavizado de RRF $k = 60$ proviene directamente del trabajo original de Cormack et al. (2009), que demostró que valores en el rango 40–80 son estables ante variaciones del corpus; valores más bajos amplifican diferencias de rango en la cabeza de la lista, y valores más altos las suavizan en exceso. Los pesos de fusión $w_{\text{BM25}} = 0,3$ y $w_{\text{vector}} = 0,7$ reflejan que el corpus incluye documentación en inglés consultada con frecuencia en español: el componente vectorial, respaldado por el modelo multilingüe BAAI/bge-m3, tiene mayor capacidad de resolver esa brecha lingüística y por eso recibe mayor peso. El componente léxico retiene peso suficiente para no perder la ventaja de BM25 ante identificadores exactos.

Los parámetros de BM25 $k_1 = 1,5$ y $b = 0,75$ corresponden a los valores canónicos recomendados por Robertson & Zaragoza (2009) para colecciones de documentación técnica, donde la frecuencia de término importa pero los documentos tienen longitudes heterogéneas. El tamaño de fragmento de 256 palabras con 32 de solapamiento fue elegido porque mantiene suficiente contexto para que el generador entienda el pasaje, sin superar el rango en el que los modelos de embeddings empiezan a perder coherencia semántica al codificar fragmentos largos.

Los parámetros de HNSW ($M = 32$, $ef_construction = 200$, $ef_search = 50$) corresponden a la configuración de alta calidad del índice: $M = 32$ produce un grafo denso que mantiene alta tasa de recuperación, y $ef_construction = 200$ asegura que la construcción del grafo explore suficientes vecinos para no crear atajos subóptimos. El valor $ef_search = 50$ equilibra exhaustividad y velocidad en tiempo de consulta.

Justificación técnica de la solución. El módulo de recuperación está diseñado para superar las deficiencias concretas de los recuperadores simples. Identificar esas deficiencias es necesario para entender por qué cada componente del módulo existe.

Un recuperador puramente léxico como BM25 falla por *desajuste de vocabulario*: si el usuario formula la consulta en español y la documentación está en inglés, o si usa sinónimos o reformulaciones del término técnico presente en el documento, BM25 no encuentra ninguna coincidencia y devuelve resultados vacíos o irrelevantes. Un recuperador puramente vectorial falla por *colapso de distinción exacta*: los modelos de embeddings proyectan términos semánticamente próximos a regiones cercanas del espacio, lo que hace que `Array.flat()` y `Array.flatMap()` aparezcan como casi equivalentes aunque el usuario esté buscando exactamente uno de los dos. Combinar ambos enfoques mediante RRF resuelve estos dos problemas sin requerir normalización de puntuaciones, porque opera únicamente sobre rangos.

Incluso con recuperación híbrida, el problema del *ranking por encoders independientes* persiste. Tanto BM25 como el modelo de embeddings puntúan consulta y fragmento por separado, sin modelar la interacción entre ellos. Un fragmento que contiene todos los términos clave puede estar mal posicionado si esos términos no aparecen en el contexto semántico correcto. El cross-encoder resuelve esto evaluando cada par (consulta, fragmento) de forma conjunta; su costo computacional se controla aplicándolo únicamente sobre los 30 candidatos preseleccionados por la recuperación híbrida.

La expansión HyDE aborda un cuarto problema: la *asimetría distribucional entre consultas y documentos*. Una consulta es corta y directiva; un documento técnico es extenso y descriptivo. El modelo de embeddings produce un vector para cada uno, pero esos vectores parten de distribuciones textuales distintas y no siempre quedan cerca en el espacio de representación aunque describan el mismo concepto. HyDE reduce esta brecha generando, antes de la búsqueda vectorial, un párrafo hipotético con la forma de un documento real que respondería la consulta. Ese párrafo es lo que se codifica y se usa como vector de consulta; BM25 continúa operando sobre la consulta original porque la coincidencia léxica no se beneficia de esta transformación.

HyDE es un mecanismo de refuerzo opcional dentro del sistema. Está desactivado por defecto porque introduce una llamada adicional al modelo generativo antes de cada recuperación, incrementando la latencia de la consulta. Se activa mediante la variable de entorno `HYDE_ENABLED=true` en escenarios donde la calidad de la recuperación vectorial es prioritaria sobre la velocidad. Si la llamada al modelo falla por cualquier razón, el sistema retrocede a la consulta original sin interrumpir el flujo ni degradar los otros componentes. HyDE no reemplaza la recuperación híbrida ni el reordenamiento: los complementa en la etapa de codificación vectorial, y su contribución es independiente de los demás.

La separación entre recuperación léxica, semántica, fusión y reordenamiento también mejora la mantenibilidad del sistema. Cada componente tiene límites claros, puede probarse de forma inde-

pendiente y puede evolucionar sin afectar a los demás. Esta arquitectura ofrece una base sólida para la integración de futuras mejoras, como modelos de embeddings más capaces, estrategias alternativas de fusión o mecanismos de retroalimentación sobre la calidad de los resultados.

Las decisiones de diseño descritas en este módulo están respaldadas por evidencia publicada. Cormack et al. (2009) demostraron que RRF supera la combinación directa de puntuaciones y los métodos de aprendizaje de ranking en una variedad de colecciones sin requerir calibración. Malkov & Yashunin (2020) establecieron HNSW como el estándar para búsqueda aproximada de vecinos en alta dimensión, superando en velocidad y tasa de recuperación a los índices planos y a las variantes IVF de FAISS para corpus de tamaño medio. Gao et al. (2023) introdujeron HyDE y mostraron que mejora la recuperación densa en escenarios de *zero-shot* sin requerir datos de relevancia etiquetados, que es exactamente la condición del sistema. El modelo **BAAI/bge-m3** ocupa consistentemente los primeros lugares en el benchmark MTEB (*Massive Text Embedding Benchmark*) para tareas de recuperación multilingüe.

Costo computacional por etapa. El pipeline de recuperación acumula costo en cada etapa. La siguiente tabla resume el costo relativo de cada componente y si es posible desactivarlo:

Etapa	Complejidad	Costo relativo	Opcional
BM25	$O(q \cdot t)$ en memoria	Bajo	No
Vectorización HNSW	$O(\log n)$ aprox.	Bajo	No
RRF (fusión)	$O(c)$ sobre candidatos	Despreciable	No
Cross-encoder (30 pares)	$O(c \cdot L)$ por par	Medio	Sí (ENV)
HyDE (llamada al LLM)	Una inferencia generativa	Alto	Sí (off por defecto)

En la configuración por defecto, las etapas costosas en orden son el cross-encoder (activo) y la vectorización de la consulta. HyDE, al estar desactivado por defecto, no contribuye al costo nominal del pipeline. El cross-encoder opera sobre un máximo fijo de 30 pares independientemente del tamaño del corpus, lo que mantiene su costo acotado y predecible.

Limitaciones del módulo. El diseño del módulo tiene limitaciones conocidas que acotan su alcance.

El tokenizador de BM25 aplica *stopwords* y *stemming* en inglés (`SnowballStemmer, language='english'`). Consultas formuladas exclusivamente en español son procesadas correctamente por el componente vectorial, que utiliza un modelo multilingüe, pero el componente léxico pierde eficiencia porque el stemmer no normaliza raíces en español y las stopwords en español no son filtradas. En la práctica, el peso vectorial dominante ($w = 0,7$) compensa esta limitación, pero documentos con términos técnicos en español podrían no ser recuperados por BM25 con la misma precisión que en inglés.

El modelo de cross-encoder **ms-marco-MiniLM-L-6-v2** fue entrenado exclusivamente sobre datos en inglés. Al reordenar pares (consulta en español, fragmento en inglés) puede degradar la posición de fragmentos relevantes que en inglés habrían sido bien posicionados. Este efecto es difícil de cuantificar sin un conjunto de evaluación etiquetado en el dominio.

La expansión HyDE genera el documento hipotético sin ningún contexto del corpus indexado. Si el modelo generativo produce un párrafo que describe un concepto inexistente en los documentos, el vector resultante puede alejar la búsqueda de los fragmentos correctos. Este riesgo aumenta cuanto más especializado es el corpus y menos representado está en los datos de entrenamiento del LLM.

El tamaño de fragmento de 256 palabras está definido de forma estática en el código y no es configurable mediante variables de entorno. Corpora con documentos de estructura muy diferente, como páginas de referencia de API cortas o guías de instalación extensas, podrían beneficiarse de tamaños de fragmento distintos.